

Gestión de citas inteligente parcial MVP Release 1 (EQUIPO 7)

Nicolás Gabriel Álvarez Aguirre, Yeison Andrés Scarpeta Diaz, Andrea Catalina Cortés
Ramírez y Dayana Sofía Mendés Perafán.

Corporación universitaria del Huila (Corhuila)

Facultad de ingeniería de sistemas

Jesús Ariel González Bonilla

Neiva, Huila

9 de Marzo del 2026

Introducción

El presente informe documenta el desarrollo y entrega del Release 1 del proyecto Gestión de Citas Inteligente.

El sistema tiene como objetivo ofrecer una plataforma modular para que negocios de servicios como salones, barberías, clínicas y veterinarias puedan gestionar sus citas de forma eficiente. Para ello se adoptó una arquitectura basada en microservicios, donde cada componente del sistema opera de forma independiente y se comunica a través de un API Gateway como punto de entrada único.

Este primer release se enfoca en el microservicio de turnos, el cual implementa un CRUD completo sobre una base de datos PostgreSQL, expuesto a través de Spring Cloud Gateway. El proyecto fue desarrollado en Java 17 con Spring Boot 3, contenedorizado con Docker y gestionado mediante un monorepo público en GitHub bajo prácticas de Git Flow.

El presente documento recorre cada uno de los criterios de la lista de chequeo establecida para la evaluación del MVP, aportando la evidencia técnica correspondiente a cada ítem.

Link del repositorio: <https://github.com/sparyock/Gestion-De-Citas-Inteligente>

1. Repositorio

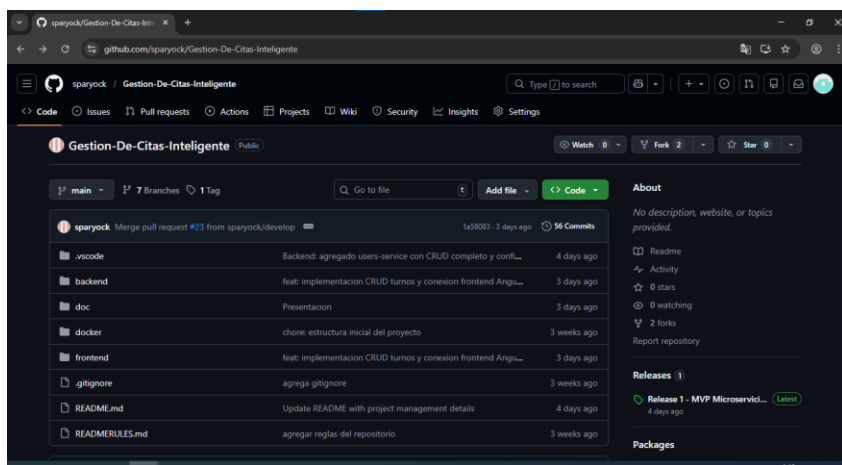
El proyecto se aloja en un repositorio público de GitHub bajo la URL github.com/sparyock/Gestion-De-Citas-Inteligente, accesible para cualquier persona sin necesidad de autenticación. Al momento de esta entrega el repositorio cuenta con 57 commits y una release oficial etiquetada como v1.0.0, publicada el 3 de marzo de 2026.

La organización del monorepo sigue una estructura clara por responsabilidad: la carpeta backend/ contiene los módulos api-gateway y user-service, la carpeta docker/ centraliza la configuración de contenedores, la carpeta doc/ agrupa la documentación técnica del proyecto, y la carpeta frontend/ alberga el microfrontend en Angular previsto para el Release 2.

El archivo README.md en la raíz del proyecto documenta el propósito del sistema, el stack tecnológico, la arquitectura del Release 1, los pasos para ejecutar el proyecto localmente y la tabla de endpoints disponibles. Adicionalmente existe un archivo READMERULES.md con las convenciones de trabajo definidas para el equipo.

El archivo .gitignore está configurado correctamente para proyectos Java con Maven, excluyendo carpetas como target/, archivos de compilación y configuraciones locales de IDE. Finalmente, la carpeta doc/ contiene la documentación técnica generada durante el desarrollo, incluyendo diagramas y descripciones del diseño del sistema.

Figura 1: Página principal del repositorio en GitHub



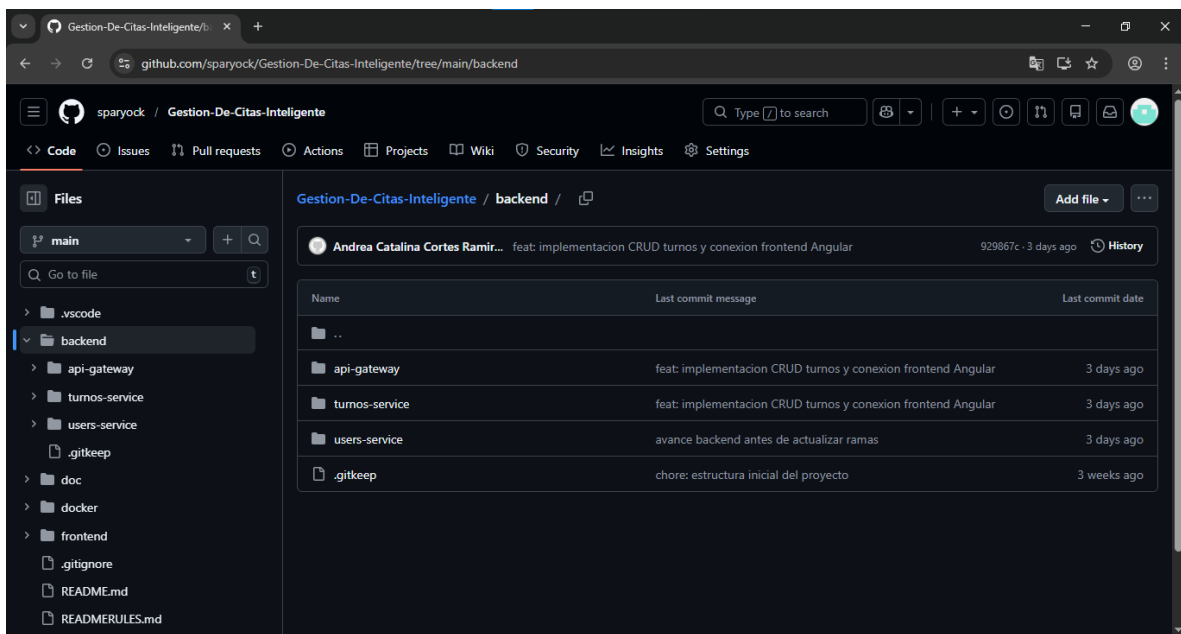
Se evidencia el repositorio público del proyecto con 56 commits y la release v1.0.0 publicada.

Figura 2: Estructura de carpetas del monorepo

backend	feat: implementacion CRUD turnos y conexion frontend Angu...	3 days ago
doc	Presentacion	3 days ago
docker	chore: estructura inicial del proyecto	3 weeks ago
frontend	feat: implementacion CRUD turnos y conexion frontend Angu...	3 days ago

Se observa la organización del proyecto en carpetas por responsabilidad: backend, docker, doc y frontend.

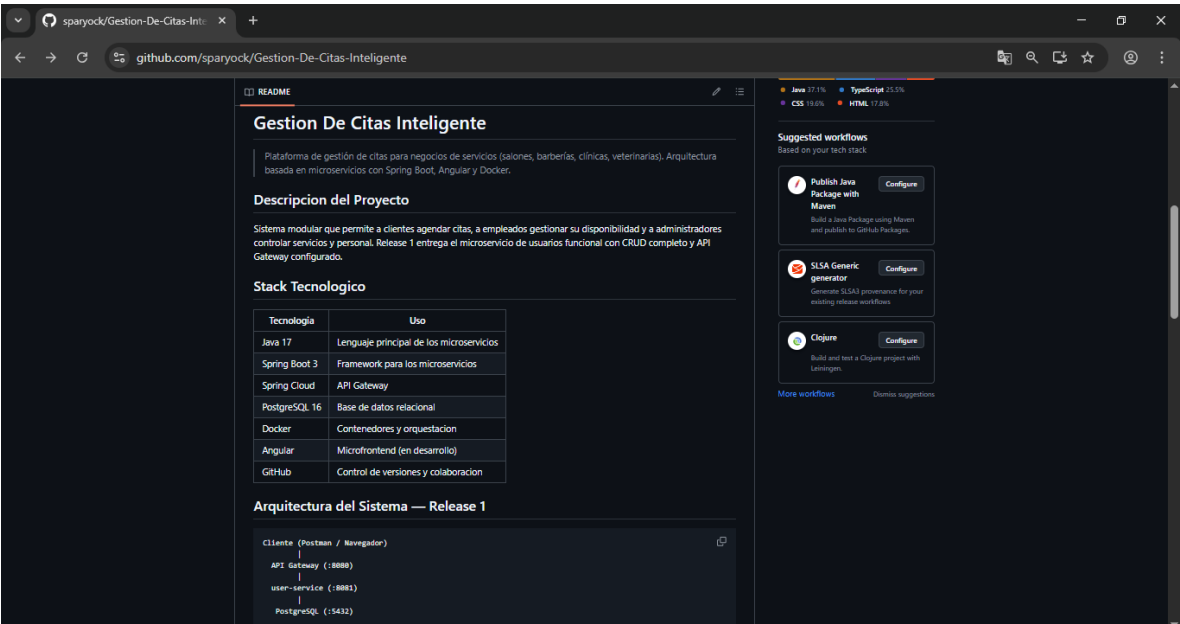
Figura 2.1: Contenido de la carpeta backend



Name	Last commit message	Last commit date
..		
api-gateway	feat: implementacion CRUD turnos y conexion frontend Angular	3 days ago
turnos-service	feat: implementacion CRUD turnos y conexion frontend Angular	3 days ago
users-service	avance backend antes de actualizar ramas	3 days ago
.gitkeep	chore: estructura inicial del proyecto	3 weeks ago

Se evidencian los tres módulos del backend: api-gateway, turnos-service y user-service.

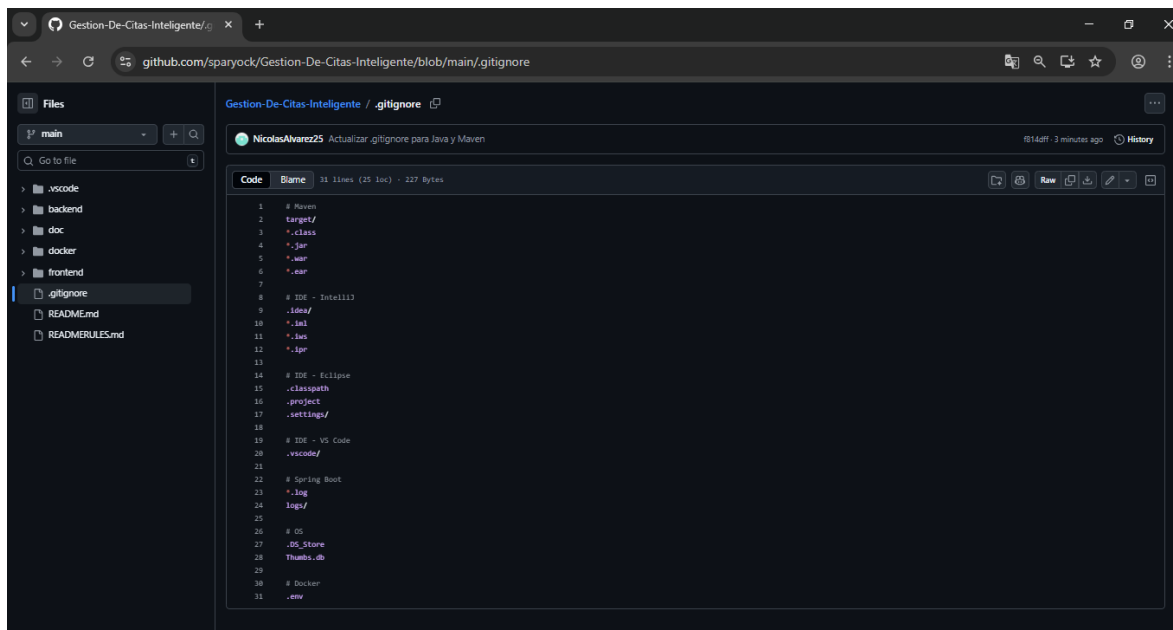
Figura 3: Contenido del README: Arquitectura y Endpoints



Endpoints Disponibles		
Metodo	URL	Descripcion
GET	http://localhost:8080/users/test	Verificar que el sistema funciona
GET	http://localhost:8080/users	Listar todos los usuarios (via Gateway)
GET	http://localhost:8080/users/{id}	Buscar usuario por ID (via Gateway)
GET	http://localhost:8081/users	Listar usuarios (directo al servicio)
GET	http://localhost:8081/users/{id}	Buscar por ID (directo al servicio)

El README documenta la arquitectura del sistema y los endpoints disponibles para el Release 1.

Figura 4: Contenido del archivo .gitignore



```
1 # Maven
2 target/
3 *.class
4 *.jar
5 *.war
6 *.ear
7
8 # IDE - IntelliJ
9 *.idea/
10 *.iml
11 *.iws
12 *.ipr
13
14 # IDE - Eclipse
15 .classpath
16 .project
17 .settings/
18
19 # IDE - VS Code
20 .vscode/
21
22 # Spring Boot
23 *.log
24 logs/
25
26 # OS
27 .DS_Store
28 Thumbs.db
29
30 # Docker
31 .env
```

El archivo .gitignore está configurado para ignorar archivos generados por Maven, IDEs de Java y el sistema operativo.

2. Git Flow

El equipo adoptó Git Flow como estrategia de control de versiones. El repositorio cuenta con las ramas principales main, develop y qa, además de una rama release.1 correspondiente a este primer hito del proyecto. Para el desarrollo de funcionalidades específicas se crearon feature branches como feature/docker-config, feature/backend-login y feature/frontend-login, siguiendo la convención de nombres acordada en el archivo READMERULES.md.

El historial de commits refleja el trabajo progresivo del equipo con 57 commits en la rama principal, con mensajes que describen los cambios realizados en cada contribución.

Durante el desarrollo se gestionaron 22 Pull Requests, todos ya mergeados, lo que evidencia un flujo de revisión de código antes de integrar cambios a las ramas principales.

Para el seguimiento de tareas se utilizaron GitHub Issues, contando con 4 issues cerrados que registran problemas y mejoras atendidos durante el desarrollo. Complementariamente el equipo utilizó un tablero en Trello para la gestión del backlog y la planificación de sprints.

Figura 5: Lista de ramas del repositorio

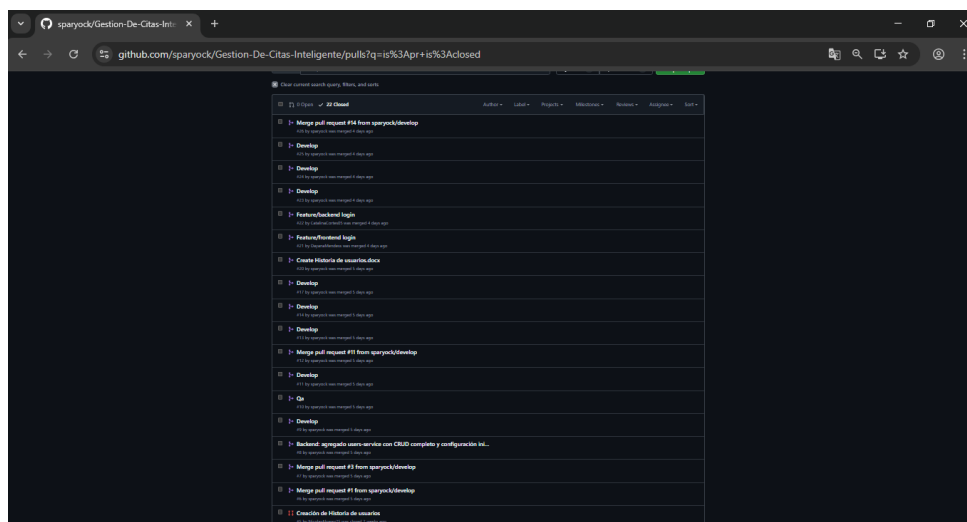
The screenshot shows the GitHub 'Branches' page for the repository 'sparyock/Gestion-De-Citas-Inteligente'. It displays a list of branches categorized into 'Default', 'Your branches', and 'Active branches'. The 'main' branch is the default. Other branches include 'feature/docker-config', 'qa', 'develop', 'release.1', 'feature/backend-login', and 'feature/frontend-login'. Each branch entry shows its update time, check status, and whether it is behind or ahead of the parent branch.

Branch	Updated	Check status	Behind	Ahead	Pull request
main	9 minutes ago			Default	
feature/docker-config	4 days ago		42	2	
qa	3 days ago		16	1	
develop	3 days ago		17	0	#224
release.1	3 days ago		19	1	
feature/backend-login	3 days ago		19	0	#222
feature/frontend-login	3 days ago		43	0	#221

Se evidencia el uso de Git Flow con ramas main, develop, qa, release.1 y feature branches.

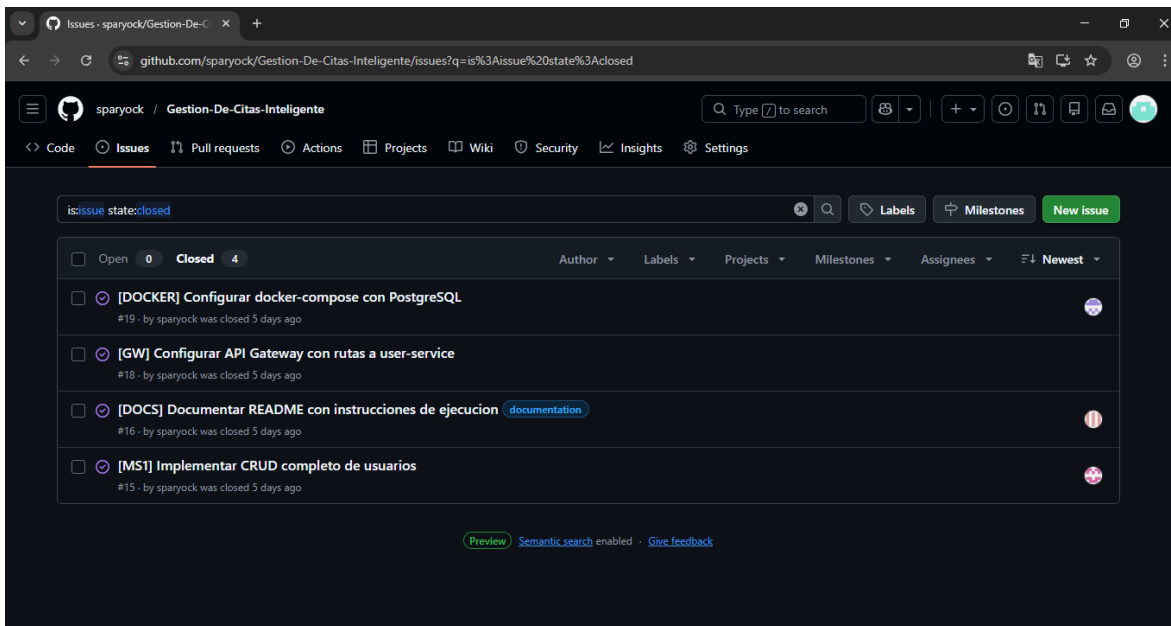
The screenshot shows the GitHub interface for the repository 'sparyock/Gestion-De-Citas-Inteligente'. The 'Commits' tab is selected, showing a list of recent commits. The first commit is 'Actualizar .gitignore para Java y Maven' by NicolasAlvarez25. Below it, several merge pull requests are listed, including 'Merge pull request #23 from sparyock/develop', 'Merge pull request #22 from sparyock/feature/backend-login', and 'Merge pull request #26 from sparyock/relase.1'. Following these are individual commits: 'Presentacion', 'feat: implementation CRUD turnos y conexion frontend Angular', 'Delete Historias de Usuario Citas.docx', 'Merge branch 'develop' into feature/backend-login', 'avance backend antes de actualizar ramas', and 'Merge pull request #21 from sparyock/feature/ frontend-login'. Each commit entry includes the commit message, the author, the time ago, a 'Verified' status, a commit hash, and icons for viewing the commit details and the associated code diff.

Figura 7: Pull Requests del proyecto



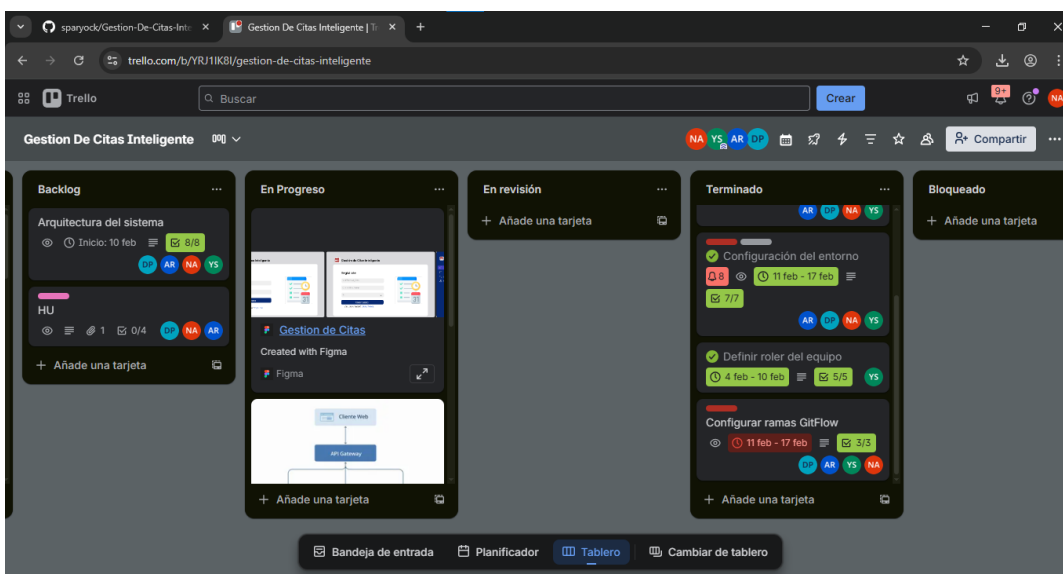
Se observa la lista de pull requests del proyecto.

Figura 8: Issues cerrados del proyecto en GitHub



Se evidencia el uso de GitHub Issues para el seguimiento de tareas durante el desarrollo del Release 1.

Figura 9: Tablero de Trello del proyecto



Se muestra el tablero de gestión de tareas utilizado por el equipo para el seguimiento del backlog y la planificación del Release 1.

3. Arquitectura

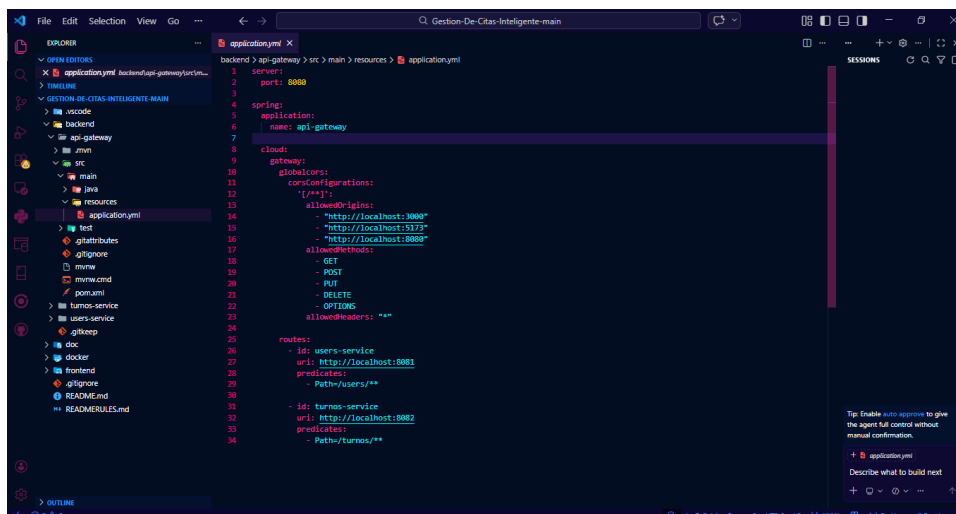
El sistema implementa una arquitectura basada en microservicios con un API Gateway como punto de entrada único. Todas las peticiones del cliente pasan primero por el Gateway configurado en el puerto 8080, el cual se encarga de enrutarlas hacia el microservicio correspondiente.

Para este Release 1 el Gateway redirige las peticiones hacia el microservicio de usuarios que corre en el puerto 8081. Esta separación permite que en futuros releases se agreguen nuevos microservicios sin afectar el punto de entrada del sistema.

El microservicio de usuarios sigue una arquitectura por capas con separación clara de responsabilidades: la capa Controller recibe las peticiones HTTP, la capa Service contiene la lógica del negocio y la capa Repository se encarga de la comunicación con la base de datos. Esta estructura facilita el mantenimiento y la escalabilidad del código.

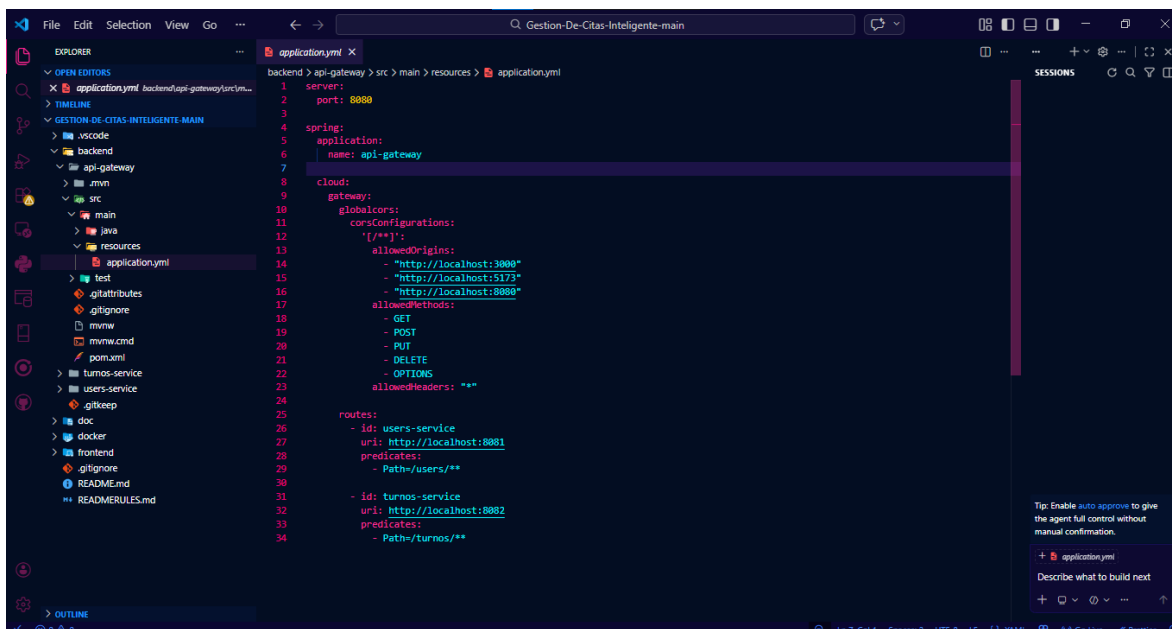
Se utilizaron DTOs para el intercambio de datos entre capas, evitando exponer directamente las entidades de la base de datos. El CRUD completo de usuarios está implementado con los métodos GET, POST, PUT y DELETE accesibles tanto directamente al microservicio como a través del Gateway.

Figura 10: Configuración del API Gateway



Se evidencia el API Gateway configurado en el puerto 8080 con Spring Cloud Gateway.

Figura 11: Enrutamiento del Gateway hacia los microservicios



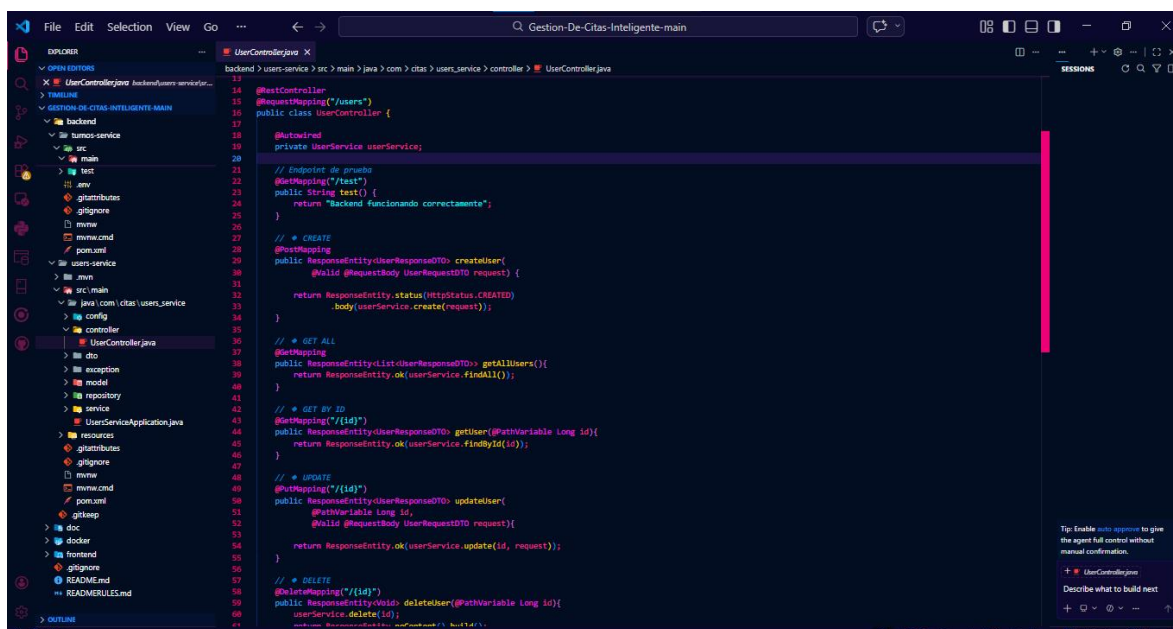
Se observa la configuración de rutas donde users redirige al user-service en el puerto 8081 y turnos al turnos-service en el puerto 8082.

Figura 12: Estructura de capas del microservicio de usuarios



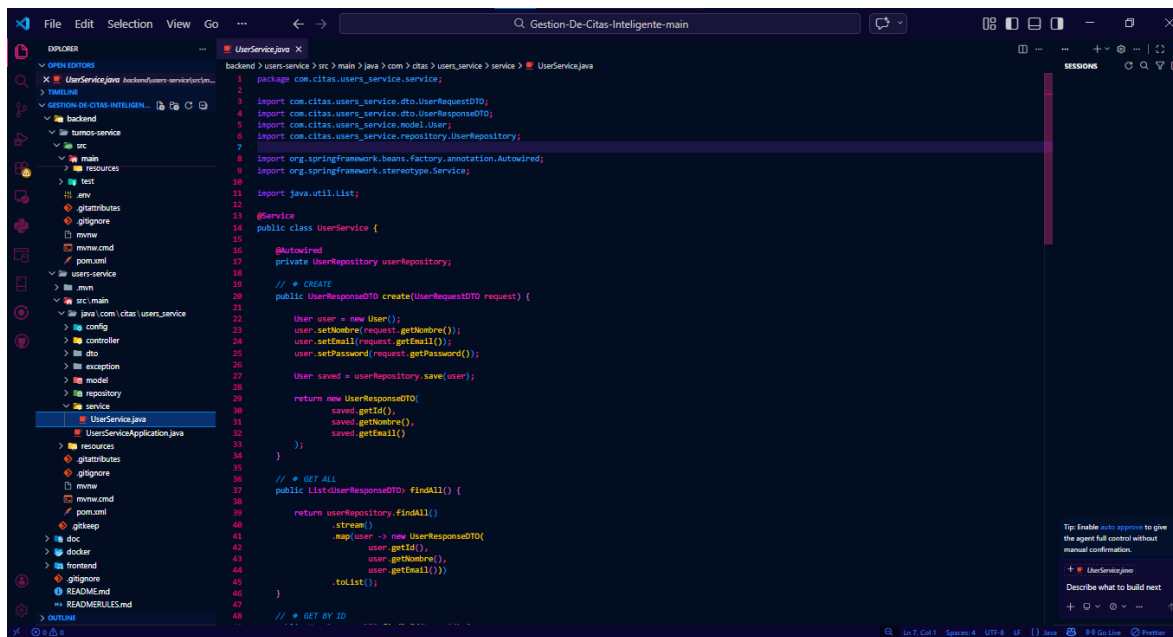
Se observa la organización del user-service en capas: controller, service, repository, dto, model, config y exception, siguiendo las buenas prácticas de Spring Boot.

Figura 13: Clase UserController del microservicio de usuarios



Se evidencia el controlador REST con los endpoints GET, POST, PUT y DELETE implementados, junto con un endpoint de prueba y login de usuarios.

Figura 14: Clase UserService del microservicio de usuarios



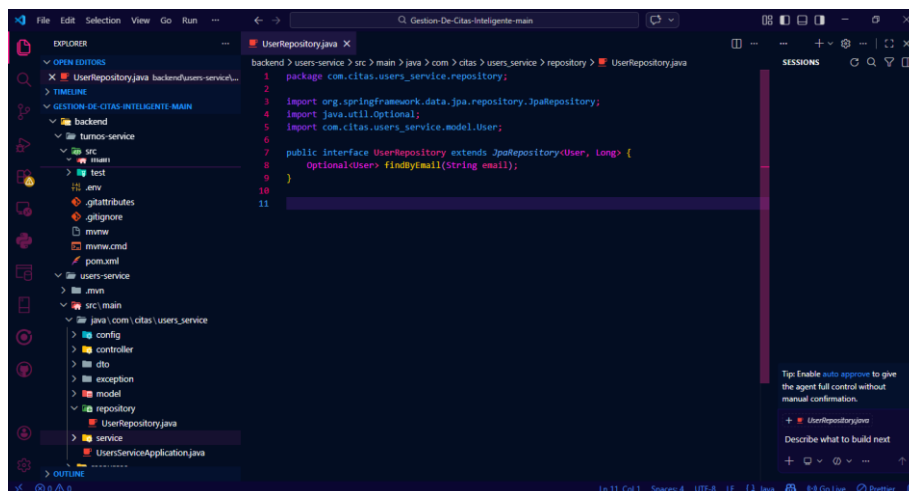
```

1 package com.citas.users_service.service;
2
3 import com.citas.users_service.dto.UserRequestDTO;
4 import com.citas.users_service.dto.UserResponseDTO;
5 import com.citas.users_service.model.User;
6 import com.citas.users_service.repository.UserRepository;
7
8 import org.springframework.beans.factory.annotation.Autowired;
9 import org.springframework.stereotype.Service;
10
11 import java.util.List;
12
13 @Service
14 public class UserService {
15
16     @Autowired
17     private UserRepository userRepository;
18
19     // + CREATE
20     public UserResponseDTO create(UserRequestDTO request) {
21         User user = new User();
22         user.setName(request.getName());
23         user.setEmail(request.getEmail());
24         user.setPassword(request.getPassword());
25         User saved = userRepository.save(user);
26         return new UserResponseDTO(
27             saved.getId(),
28             saved.getName(),
29             saved.getEmail()
30         );
31     }
32
33     // + GET ALL
34     public List<UserResponseDTO> findAll() {
35         return userRepository.findAll()
36             .stream()
37             .map(user -> new UserResponseDTO(
38                 user.getId(),
39                 user.getName(),
40                 user.getEmail()
41             ))
42             .toList();
43     }
44
45     // + GET BY ID
46
47
48

```

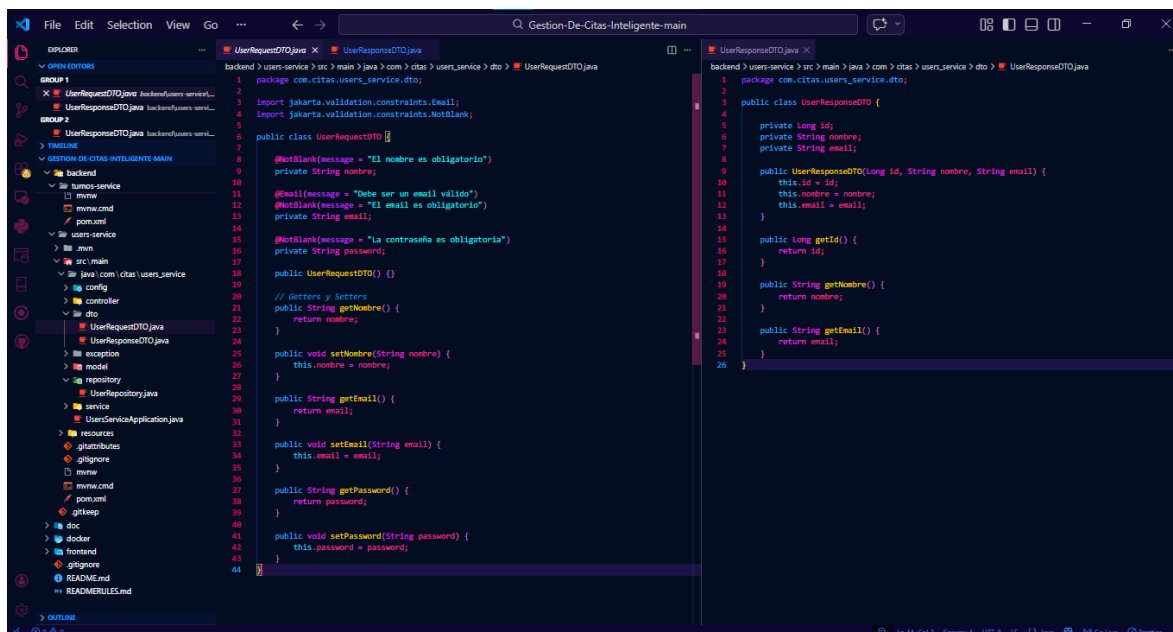
Se evidencia la capa de servicio con la lógica de negocio implementada para las operaciones CREATE, GET ALL, GET BY ID, UPDATE y DELETE.

Figura 15: Interfaz UserRepository del microservicio de usuarios



Se evidencia el repositorio extendiendo JpaRepository, lo que permite las operaciones CRUD sobre la base de datos PostgreSQL sin necesidad de implementar los métodos manualmente.

Figura 16: Clases UserRequestDTO y UserResponseDTO



Se evidencia el uso de DTOs para el intercambio de datos, separando la representación de entrada y salida de la entidad de usuario.

4. Microservicio de Usuarios

El microservicio de usuarios expone sus endpoints tanto de forma directa en el puerto 8081 como a través del API Gateway en el puerto 8080. Los endpoints disponibles permiten listar usuarios, buscar por ID, crear, actualizar y eliminar, cubriendo así el CRUD completo del sistema.

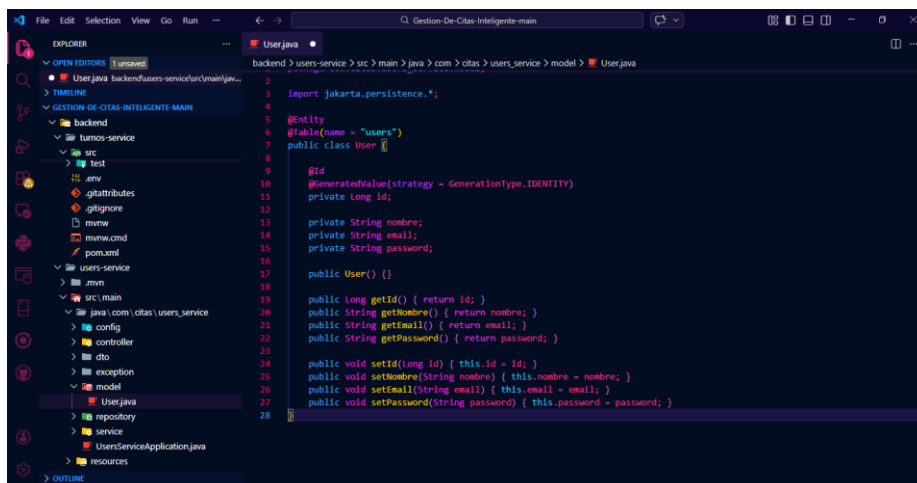
Para la persistencia de datos se utilizó PostgreSQL 16 como base de datos relacional. La entidad User está mapeada con JPA usando anotaciones como `@Entity`, `@Table`, `@Id` y `@GeneratedValue`, lo que permite que Spring Boot gestione automáticamente la creación y manipulación de la tabla en la base de datos.

Durante la demo del proyecto se realizaron pruebas funcionales con Postman sobre el microservicio de turnos, el cual comparte la misma arquitectura y configuración que el microservicio de usuarios. Ambos microservicios están configurados en el API Gateway y siguen el mismo patrón de desarrollo.

CRUD completo implementado: Esta parte está en las figuras 13 y 14.

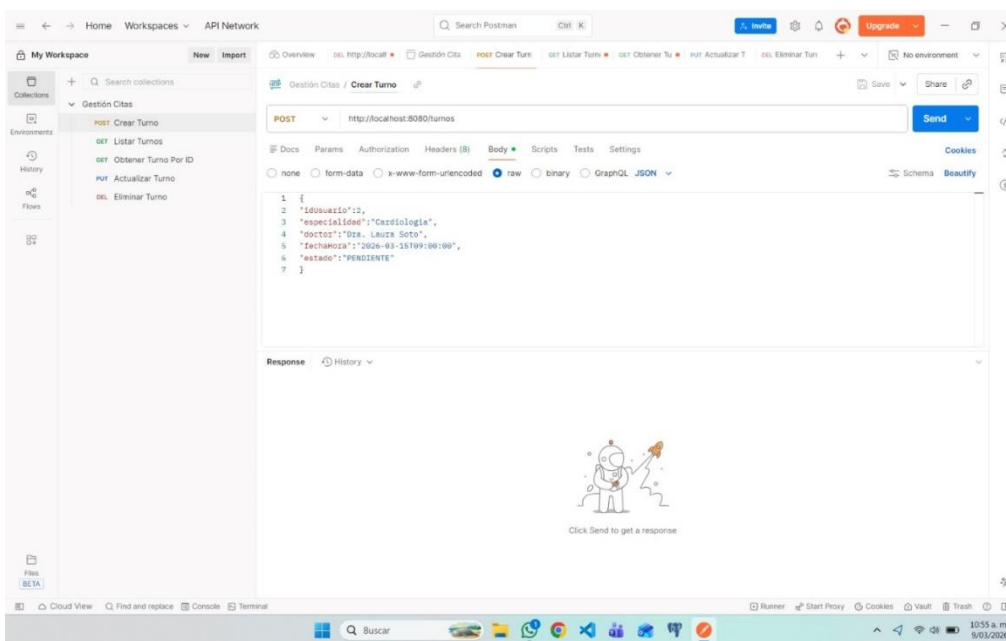
Entidad mapeada con JPA

Figura 17. Entidad User mapeada con JPA



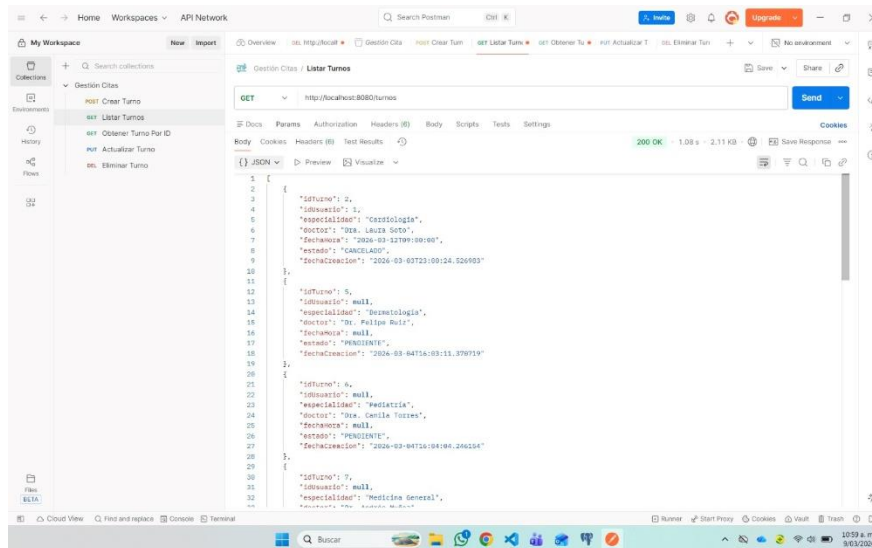
Se evidencia el mapeo de la entidad de usuario a la base de datos PostgreSQL mediante las anotaciones @Entity, @Id y @Column.

Figura 18: Endpoint POST /turnos en Postman



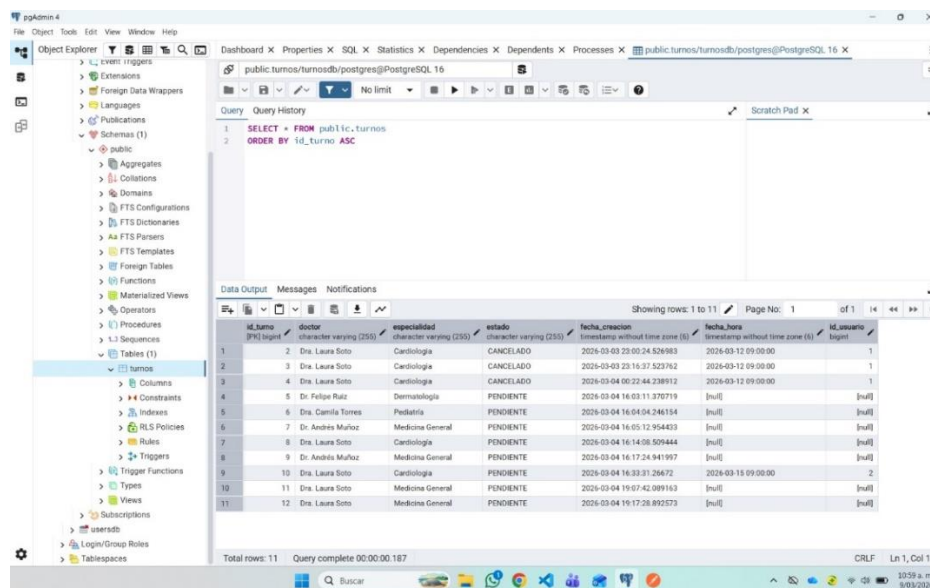
Se evidencia el endpoint para crear un turno a través del API Gateway en <http://localhost:8080/turnos>, con los campos requeridos: idUsuario, especialidad, doctor, fecha, Hora y estado.

Figura 19: Endpoint GET /turnos: respuesta exitosa via Gateway



Se evidencia la respuesta con código 200 OK al consultar todos los turnos a través del API Gateway en `http://localhost:8080/turnos`, mostrando los datos persistidos en la base de datos.

Figura 20: Base de datos en pgAdmin: tabla turnos



Se evidencia la persistencia de datos en PostgreSQL 16 con 11 registros en la tabla turnos, mostrando campos como doctor, especialidad, estado, fecha de creación y fecha de la cita.

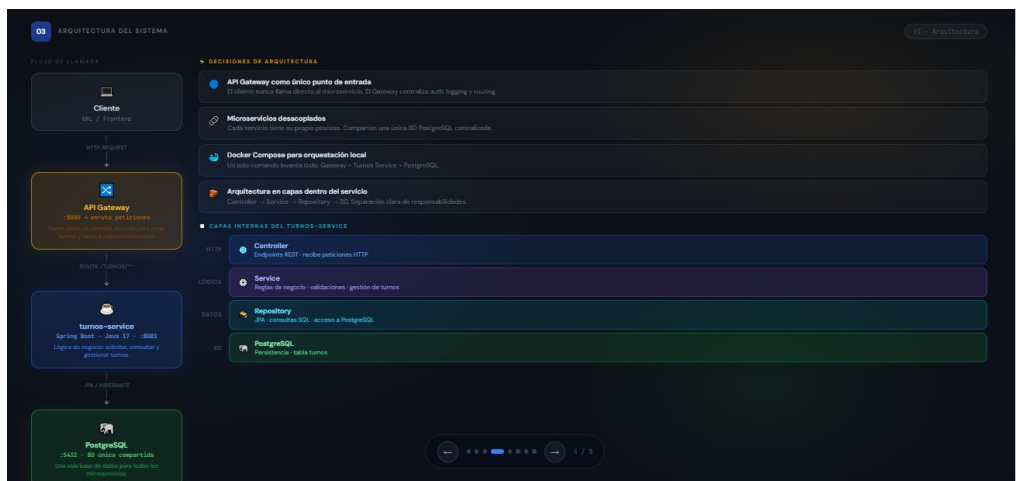
5. Demo

El sistema fue presentado ante el docente y compañeros de clase, mostrando el funcionamiento del microservicio de turnos como evidencia del avance del Release 1. Durante la demo se explicó el problema que resuelve el sistema: la dificultad que tienen los negocios de servicios como barberías, clínicas y veterinarias para gestionar sus citas de forma organizada y eficiente.

Se presentó el diagrama de arquitectura del sistema explicando el flujo de las peticiones desde el cliente hasta la base de datos pasando por el API Gateway. Se realizaron pruebas en vivo con Postman demostrando la creación y consulta de turnos a través del Gateway en el puerto 8080, con respuestas exitosas y datos persistidos en PostgreSQL.

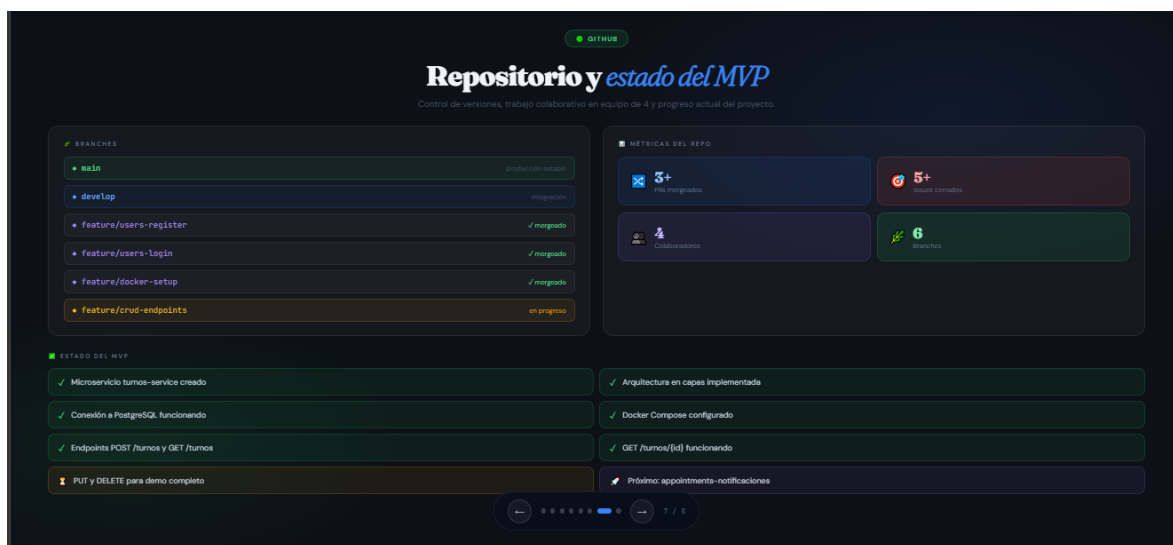
El repositorio fue mostrado en GitHub evidenciando la estructura del monorepo, las ramas de trabajo y el historial de commits del equipo.

Figura 21: Diagrama de arquitectura del sistema



Se presenta la arquitectura del sistema mostrando el flujo de llamadas desde el cliente hasta PostgreSQL, pasando por el API Gateway y el turnos-service, con las capas internas Controller, Service, Repository y Base de datos.

Figura 22: Repositorio y estado del MVP presentado en la demo



Se muestra el estado del repositorio durante la presentación: ramas main, develop y feature branches mergeadas, junto con las métricas del repo y el estado de los entregables del MVP.

Conclusiones

Durante el desarrollo del Release 1 el equipo logró construir una base sólida para el sistema de gestión de citas inteligente. Se implementó el microservicio de usuarios con CRUD completo y se configuró el API Gateway como punto de entrada único. En la demo se presentó el funcionamiento del microservicio de turnos, evidenciando el avance del proyecto más allá de lo planificado para este release.

El uso de Git Flow permitió organizar el trabajo del equipo de forma ordenada, con ramas diferenciadas para desarrollo, pruebas y funcionalidades específicas. El historial de commits y los Pull Requests mergeados evidencian un proceso de trabajo colaborativo y controlado.

La arquitectura por capas implementada en Spring Boot facilita el mantenimiento del código y sienta las bases para agregar nuevos microservicios en los próximos releases. La contenedorización con Docker y el uso de PostgreSQL como base de datos garantizan la portabilidad y escalabilidad del sistema.

Referencias

Repositorio principal: <https://github.com/sparyock/Gestion-De-Citas-Inteligente>

Tablero de gestión: <https://trello.com/b/YRJ1IK8I/gestion-de-citas-inteligente>